

ĐÁP ÁN DÀNH CHO MC (Không in phần này lên phiếu)

Khi các nhóm nộp bài, bạn hãy đối chiếu với đáp án sau và giải thích cho lớp hiểu:

ĐÁP ÁN CÂU 1: 4 LỖI LOGIC NGHIỆP VỤ (INVARIANTS)

1. **Lỗi Khóa ngoại ảo (Orphaned Data):** Tại `ORD_002`, mã sản phẩm `"PROD_003"` không hề tồn tại trong kho (Phần 2).
2. **Lỗi Ràng buộc Biên (Missing Boundary):** Cả `ORD_002` và `ORD_003` đều mất khối object `"shippingAddress"`, sẽ làm crash frontend khi render.
3. **Lỗi Logic Tài chính:** Tại `ORD_003`, tổng tiền `totalPrice` là `9999` (Tính đúng phải là: $1500 + 100 + 0 = 1600$).
4. **Lỗi Bất đồng bộ Trạng thái:** Tại `ORD_003`, `"isPaid": true` nhưng hoàn toàn thiếu trường `"paidAt"`.

ĐÁP ÁN CÂU 2: CÁC ĐIỂM CẦN DATA MASKING

Các nhóm phải khoanh được 3 mục tiêu sau. Nếu nhóm nào giải thích được **công cụ nào dùng cho mục tiêu nào**, hãy cho điểm tuyệt đối:

1. Các trường phẳng (Flat Fields) trong Users:

- **Mục tiêu:** `"email"` và `"name"` của `USER_88`, `USER_99`.
- **Cách xử lý:** Dùng **Faker.js** để che ngẫu nhiên với tốc độ cao (không cần AI).

2. Khối dữ liệu cấu trúc lồng (Nested Object):

- **Mục tiêu:** Toàn bộ object `"shippingAddress"` trong `ORD_001`.
- **Cách xử lý:** Dùng **Gemini AI** để sinh địa chỉ giả tại Việt Nam nhưng vẫn giữ đúng keys (`address`, `city`, `postalCode`) tránh làm crash ứng dụng.

3. Trường văn bản tự do (Free-text) chứa PII ẩn:

- **Mục tiêu:** Trường `"notes"`. Tại `ORD_001` lộ Tên, SĐT, Mã cổng. Tại `ORD_003` lộ thông tin thẻ tín dụng.
- **Cách xử lý:** ĐÂY LÀ ĐIỂM ĂN TIỀN CỦA AI! Các tool mask truyền thống (như Regex) rất khó bắt chính xác ngữ cảnh đoạn text tiếng Việt này. **Gemini AI** sẽ đọc và tự động làm mờ: *"Giao cho anh [MASKED], gọi số [MASKED]... Cổng nhà mã số [MASKED]"* trong khi vẫn giữ nguyên ý nghĩa *"Nhờ bảo vệ chung cư nhận hộ"* ở `ORD_002` vì câu này an toàn.